UT2 – Parte 1
Estrategias de resolución de problemas

Módulo – Modelos de Inteligencia Artificial
Curso de especialización en Inteligencia Artificial y Big Data

Contenido original: CI María Ana Sanz
Adaptado por: Encarna Delgado. IES Arcipreste de Hita

Contenidos UT2

1. Sistemas de Resolución de Problemas (IA Simbólica):
    1. Búsqueda en espacio de estados.
        1. Búsqueda Ciega o No informada.
        2. Búsqueda heurística.
        3. Búsqueda local.
2. Sistemas Basados en Conocimientos (SBC) (Parte II)
    1. Introducción a Sistemas basados en reglas.
    2. Sistemas de razonamiento impreciso.

Introducción

- **Inteligencia** => Capacidad para **resolver problemas**.
- **Resolución de problemas**: proceso que partiendo de unos datos iniciales y utilizando un conjunto de procedimientos, es capaz de determinar el conjunto de pasos o elementos que nos llevan a lo que denominaremos una solución.

- Pasos generales para resolución de problemas:
  1. Partimos de la descripción en lenguaje natural del problema y de los pasos para resolverlo.
  2. Abstracción: Expresar el problema con un lenguaje formal y estructurado.
  3. Definir algoritmos o estrategias que alcancen la solución usando dicho lenguaje formal.

Tipos de soluciones: una solución cualquiera, todas las soluciones, la solución más corta, la solución menos costosa, etc.

Conceptos fundamentales

Elementos fundamentales de un sistema de resolución de problemas:

1. Un punto de partida o **estado inicial**.

2. Un objetivo a alcanzar o **estado objetivo**.

3. Un conjunto de **acciones** o herramientas a nuestra disposición para llegar desde el punto de partida hasta el objetivo.

4. Una serie de **restricciones** sobre el objetivo y los pasos a seguir para alcanzarlo.

5. Elementos que son relevantes en el problema definidos por el dominio concreto: qué **conocimiento** tenemos que nos puede ayudar a resolver el problema de una manera eficiente.

Alternativas de representación

Dependiendo de la naturaleza del problema elegiremos distintas alternativas de representación

1. Problemas generales
   a. **Espacio de estados;** Definir todos los estados posibles y las acciones para pasar de una a otra (operadores)
   b. Reducción a subproblemas: dividir el problema en partes pequeñas y resolverlas de forma independiente (como en la programación modular)

2. Tipos de problemas más específicos:
   c. **Satisfacción de restricciones:** combinación de valores que cumpla todas las reglas impuestas (horarios escolares)
   d. **Juegos (búsqueda con adversario/s):** variedad de los espacios de estados con otro agente que intenta impedir que alcancemos la meta

Espacio de estados

Denominaremos **Estado** a la representación de los elementos que describen el problema en un momento

- Podemos definir un problema por los elementos que intervienen y sus relaciones.
- En cada instante de la resolución de un problema esos elementos tendrán unas características y relaciones específicas
- Distinguiremos dos estados especiales el **Estado Inicial** (punto de partida) y el **Estado Final** (objetivo del problema)
- Debemos decidir qué información incluimos en el estado para que el sistema no se vuelva ineficiente.

Espacio de estados

Para poder movernos entre los diferentes estados necesitamos **operadores de transformación**.

- **Operador**: Función de transformación sobre la representación de un estado que lo convierte en otro estado
- Para representar un estado necesitamos:
  - Condiciones de aplicabilidad: requisitos que deben cumplirse para que la acción sea válida
  - Función de transformación: mecanismo que describe qué cambia cuando se aplica el operador
- ¿Que operadores? ¿Cuantos? ¿Que granularidad?

Espacio de estados

- Los estados y su relación de accesibilidad conforman lo que se denomina **espacio de estados**
- Representa todos los caminos que hay entre todos los estados posibles de un problema

- **Solución**: Puede requerirse la secuencia de pasos que llevan del estado inicial al final (secuencia de operadores) o simplemente el propio estado final.
- **Tipos de solución:** una cualquiera, la mejor, todas.
- **Coste de una solución:** Gasto en recursos de la aplicación de los operadores a los estados u otros criterios de coste. Puede ser importante o no según el problema y que tipo de solución busquemos.

Espacio de estados - Aplicaciones

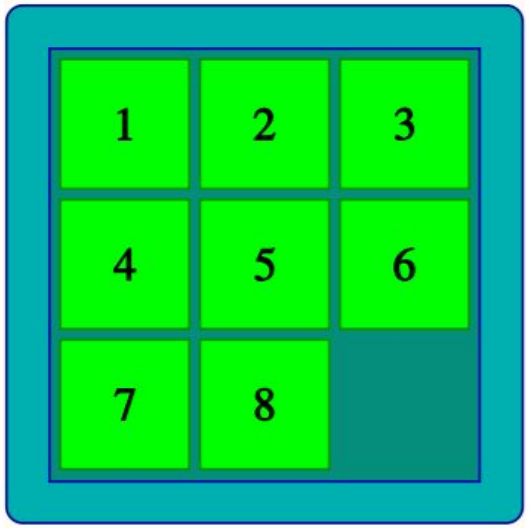
Problemas de la vida real que se pueden plantear y resolver como espacios de estados

- Búsqueda de rutas en redes informáticas
- Rutas aéreas para viajar
- Problema del viajante
- Diseño de microchips
- Ensamblaje de componentes
- Desplazamiento de robots
- Problemas de asignación de recursos

Espacio de estados - Pasos básicos

1. Definir el conjunto de estados del problema (explícita o implícitamente)
2. Especificar el estado inicial
3. Especificar el estado final o las condiciones que cumple
4. Especificar los operadores de cambio de estado (condiciones de aplicabilidad y función de transformación)
5. Especificar el tipo de solución:
   a. La secuencia de operadores o el estado final
   b. Una solución cualquiera, la mejor (definición de coste),...

Espacio de estados - Puzzle 8



- Espacio de estados: Configuraciones de 8 fichas en el tablero
- Estado inicial: Cualquier configuración
- Estado final: Fichas en orden específico
- Operador:
  - **Mover hueco**
  - Condiciones: El movimiento está dentro del tablero
  - Transformación: Intercambio entre el hueco y la ficha en la posición del movimiento
- Solución: Qué pasos + El menor número

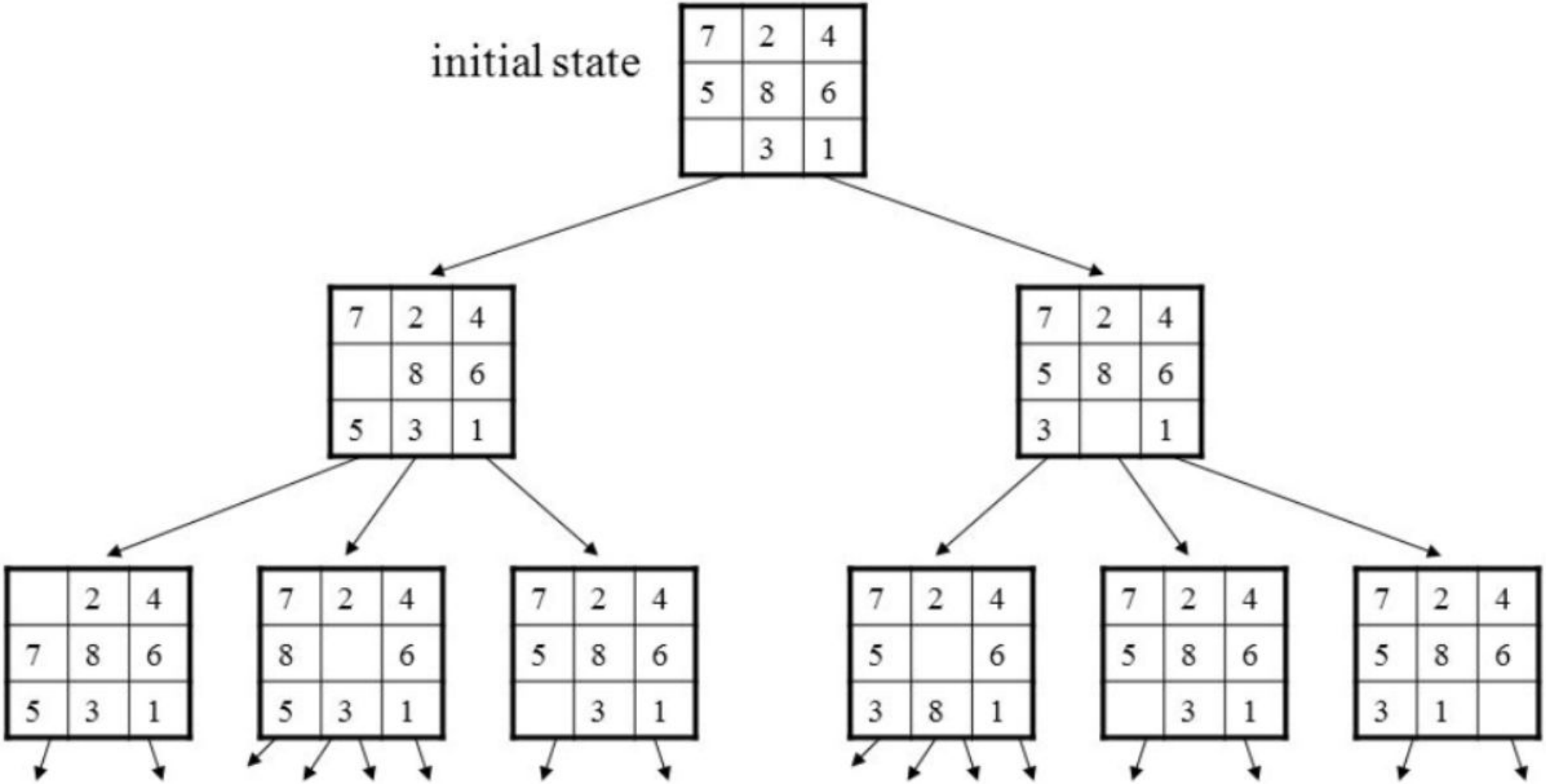
Espacio de estados - Representación

- Primero definiremos una representación del espacio de estados para poder implementar algoritmos que busquen soluciones
- Estructuras de datos: Árboles y Grafos
- Estados = Nodos
- Operadores = Arcos entre nodos (dirigidos)
- Árboles: Solo un camino lleva a un nodo
- Grafos: Varios caminos pueden llevar a un nodo

- El espacio de estados **puede ser infinito**
- En esos casos construimos el árbol a medida que lo generamos.
- Si no importa el camino, sino la solución final, podemos ir borrando los nodos recorridos.

Espacio de estados - Representación

Características a analizar:
Cardinalidad del 8-puzzle: 9! = 362.881 estados.
Factor de ramificación: entre 4 y 2.



Espacios de estados - Algoritmos

Los espacios de estados son la base fundamental para todos los algoritmos de resolución de problemas

Algoritmos de búsqueda Ciega o No informada

- **No dependen de información propia del problema.**
- No tienen en cuenta el coste de la solución en la búsqueda
- **Funcionamiento sistemático**, siguen un orden de visitas y generación de nodos establecido por la estructura del espacio de búsqueda
- Anchura prioritaria, Profundidad prioritaria y variantes.

Algoritmos de Búsqueda heurística y de Búsqueda local

- **Se sirven del conocimiento del problema en cuestión.**
- Utilizan una estimación del coste de la solución para guiar la búsqueda
- No siempre garantizan el óptimo, ni una solución
- Hill-climbing, Branch and Bound, A$*$, IDA$*$

Espacio de estados - Algoritmo general

El espacio de estados se representa mediante **Árboles** (un solo camino a cada nodo) o **Grafos** (múltiples caminos hacia un mismo estado).

- **Generación Dinámica:** En la mayoría de los problemas, el espacio de estados puede ser inmenso o incluso infinito. Por ello, **no se crean todos los estados desde el principio**, sino que el árbol se construye a medida que el algoritmo los genera

- **Nodos Abiertos:** Es la "frontera" de la búsqueda. Estados que ya hemos descubierto pero que aún no han sido explorados.
- **Nodos Cerrados:** Son los estados ya visitados y expandidos. Almacenarlos evita que el algoritmo entre en bucles infinitos al repetir caminos en un grafo.
- **Optimización de Memoria:** Si no nos importa el camino seguido, sino solo alcanzar la meta, los nodos ya visitados pueden ir borrándose o descartándose para ahorrar recursos

Espacio de estados - Algoritmo general

Algoritmo general

```
funcion Busqueda_en_espacio_de_estados retorna solucion
    Seleccionar el primer estado como el estado actual
    mientras el estado actual no es el estado final hacer
        Generar y guardar sucesores del estado actual ( expansion )
        Escoger el siguiente estado entre los pendientes ( seleccion )
    fmientras
    retorna solucion
ffuncion
```

Operaciones básicas:

- Expansión.
- Selección.

Espacio de estados - Algoritmo detallado

Algoritmo general más detallado

```
Algoritmo Busqueda General
    Est_abiertos.insertar(Estado Inicial)
    Actual= Est_abiertos.primero( )
    mientras no es_ final( Actual ) y no Est_abiertos.vacia( ) hacer
        Est_abiertos.borrar_primero( )
        Est_cerrados.insertar ( Actual )
        Hijos= generar_sucesores ( Actual )
        Hijos= tratar_repetidos( Hijos , Est_cerrados , Est_abiertos )
        Est_abiertos.insertar( Hijos )
        Actual= Est_abiertos.primero( )
    Fmientras
fAlgoritmo
```

Nodos Abiertos: nodos generados pero aún no visitados y expandidos.
Nodos Cerrados: ya visitados y expandidos. Interesante si usamos grafos, pero implica un mayor consumo de memoria.

Espacios de estados - Propiedades

Propiedades de los algoritmos de búsqueda (para analizarlos y compararlos):

- **Completitud**: Completo si siempre llega a una solución.
- **Optimalidad**: Si alcanza la mejor solución según ciertos criterios.
- **Complejidad temporal**: Coste de la búsqueda en función del tamaño del problema.
- **Complejidad espacial**: espacio de memoria requerido para almacenar los nodos a explorar y/o ya explorados.

Espacios de estados - Conceptos

Los siguientes factores son necesarios para entender cómo calculamos la complejidad:

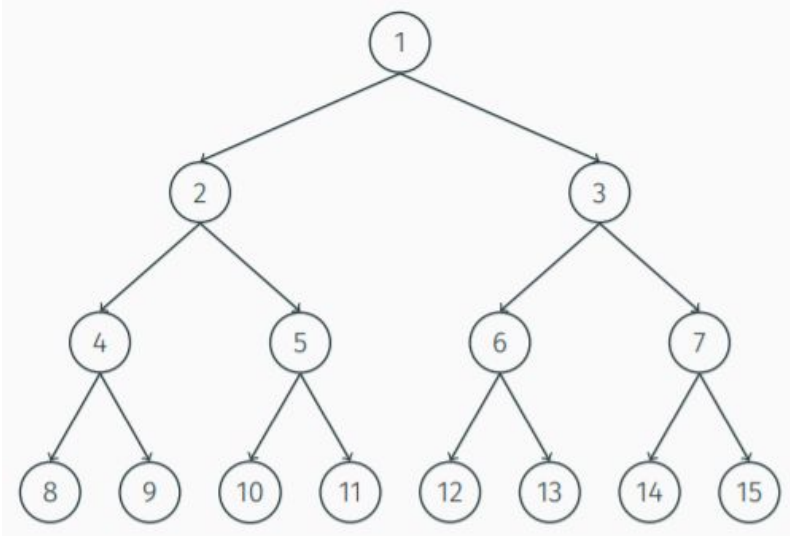
- **Factor de ramificación:** Número máximo de sucesores o hijos que puede tener un estado. (r)

- **Profundidad de la solución**: número de pasos o niveles desde el inicio hasta el fin (p)

- **Coste:** valor que representa el **gasto de recursos** asociado a la aplicación de los operadores para pasar de un estado a otro

Primero en Anchura (BFS)

- Los nodos se visitan y generan por niveles.
- La estructura para los nodos abiertos es una cola (FIFO).
- Un nodo es visitado cuando todos los nodos de los niveles superiores y sus hermanos precedentes han sido visitados

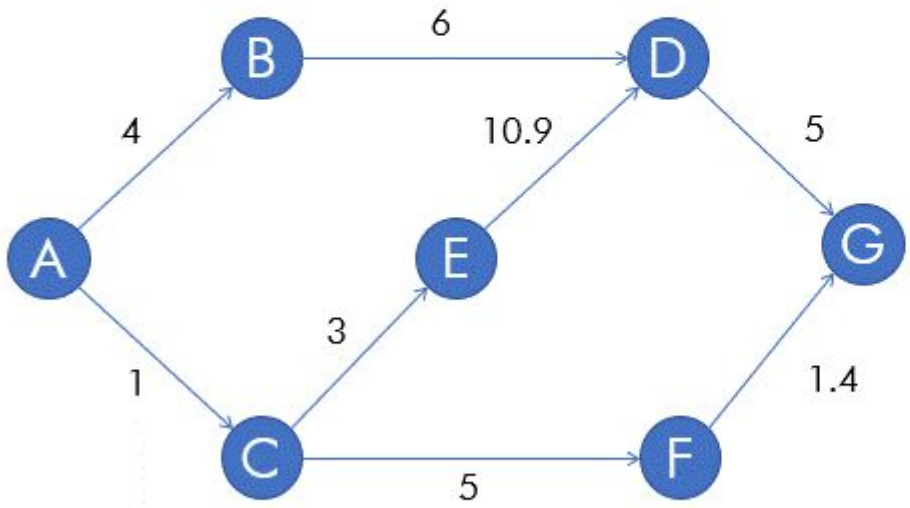


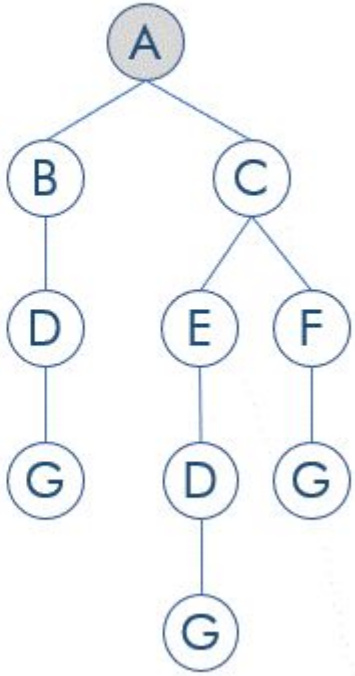
Características:

- **Completitud:** El algoritmo siempre encuentra una solución
- **Complejidad temporal:** Exponencial respecto al factor de ramificación y la profundidad de la solución $O(r^p)$
- **Complejidad espacial:** Exponencial respecto al factor de ramificación y la profundidad de la solución $O(r^{p+1})$. Si la profundidad de la solución es p, antes de verificarlo expande todos los nodos de su nivel.
- **Optimalidad:** La solución que se encuentra es óptima en número de niveles desde la raíz

Primero en Anchura (BFS)

BUSQUEDA EN ANCHURA (BFS)





Primero en Anchura (BFS)

Si el nivel al que se halla la solución es alto (profundo), es probable que el tiempo necesario para hallar la solución y su consumo de memoria sean excesivos.

Profundidad	Nodos~	Tiempo	Espacio
2	**110**	0.11 ms.	107 Kb
4	**11110**	11 ms.	10.6 Mb
6	$\mathbf{10^6}$	1.1 s.	1 Gb
8	$\mathbf{10^8}$	2 min.	103 Gb
10	$\mathbf{10^{10}}$	3 horas	10 Tb
12	$\mathbf{10^{12}}$	13 días	1 Pb
14	$\mathbf{10^{14}}$	3.5 años	99 Pb
16	$\mathbf{10^{16}}$	350 años	10 Eb

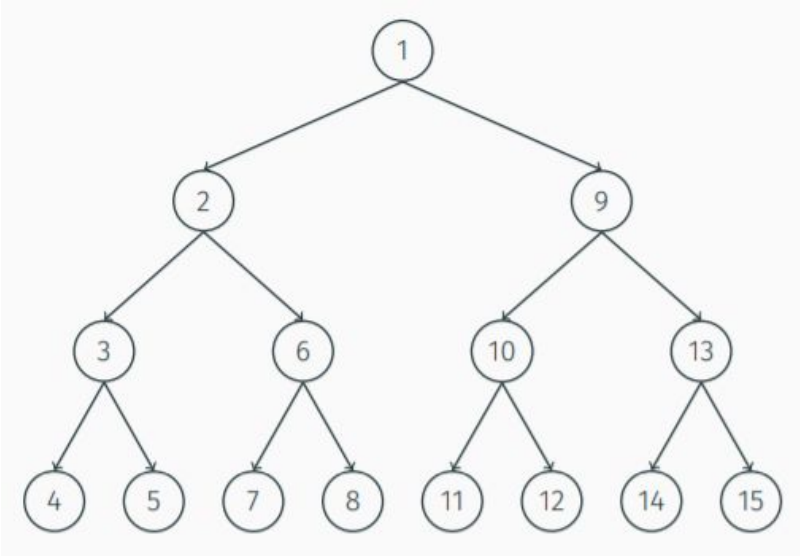
Ejemplo suponiendo que el factor de ramificación medio es 10, que se recorren 10^6 nodos por seg. y un consumo en memoria de 1000 bytes por nodo

Primero en Anchura (BFS)

```
Procedimento: Busqueda en anchura ()
    Est_abiertos.insertar(Estado inicial, final)
    Actual ← Est_abiertos.primero()
    mientras no es_final(Actual) y no Est_abiertos.vacia() hacer
        Est_abiertos.borrar_primero()
        Est_cerrados.insertar(Actual)
        Hijos ← generar_sucesores (Actual)
        Hijos ← tratar_repetidos(Hijos, Est_cerrados, Est_abiertos)
        Est_abiertos.insertar(Hijos, final)
        Actual ← Est_abiertos.primero()
    Fin mientras
Fin procedimiento
```

Primero en Profundidad (DFS)

- Los nodos se visitan y generan buscando los nodos a mayor profundidad y retrocediendo cuando no se encuentran nodos sucesores
- La estructura para los nodos abiertos es una pila (LIFO).
- Para garantizar que el algoritmo acaba debe imponerse un límite en la profundidad de exploración.

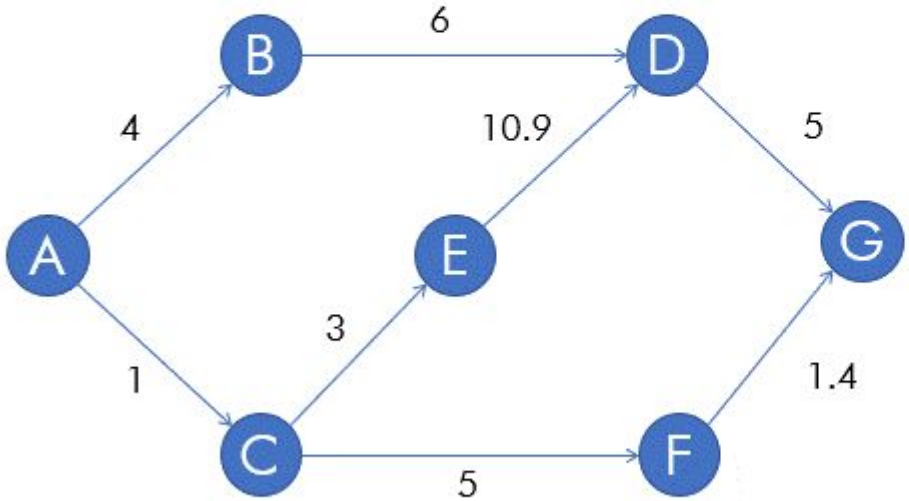


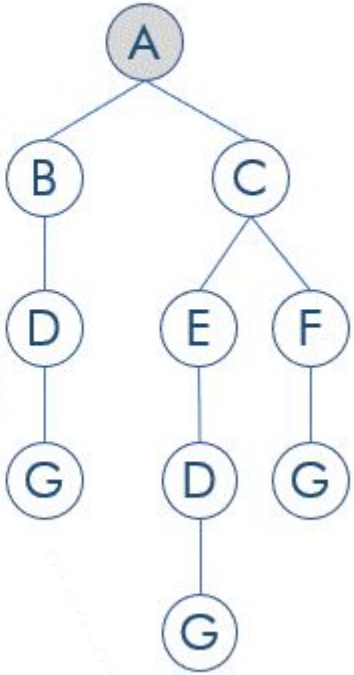
Características:

- **Completitud:** El algoritmo encuentra una solución si se impone un límite de profundidad y existe una solución dentro de ese límite.
- **Complejidad temporal:** Exponencial respecto al factor de ramificación y la profundidad del límite de exploración $O(r^p)$
- **Complejidad espacial:** En el caso de no controlar los nodos repetidos el coste es lineal respecto al factor de ramificación y el límite de profundidad O(rp). Si tratamos repetidos el coste es igual que en anchura.
- **Optimalidad:** No se garantiza que la solución sea óptima.

Primero en Profundidad (DFS)

BUSQUEDA EN PROFUNDIDAD (DFS)





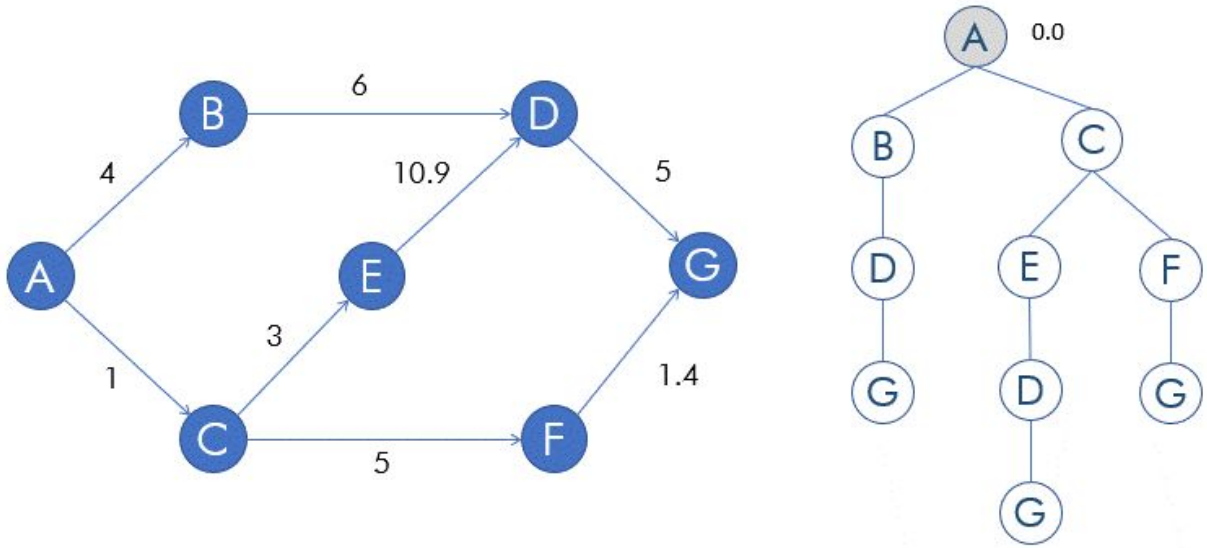
Primero en Profundidad (DFS)

```
Procedimento: Busqueda en profundidad limitada (limite: entero)
    Est_abiertos.insertar(Estado inicial, inicio)
    Actual ← Est_abiertos.primero()
    mientras no es_final(Actual) y no Est_abiertos.vacia() hacer
        Est_abiertos.borrar_primero()
        Est_cerrados.insertar(Actual)
        si profundidad(Actual) ≤ límite entonces
            Hijos ← generar_sucesores (Actual)
            Hijos ← tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)
            Est_abiertos.insertar(Hijos, inicio)
        sino
            Actual ← Est_abiertos.primero()
        Fin si
    Fin mientras
Fin procedimiento
```

Búsqueda de Coste Uniforme (UCS)

- Al expandir cada nodo calcula el coste de ir desde el nodo actual a cada uno de sus sucesores.
- Se comporta como el algoritmo BFS pero usando el coste acumulado de cada nodo como prioridad, expandiendo siempre el nodo de menor coste calculado.

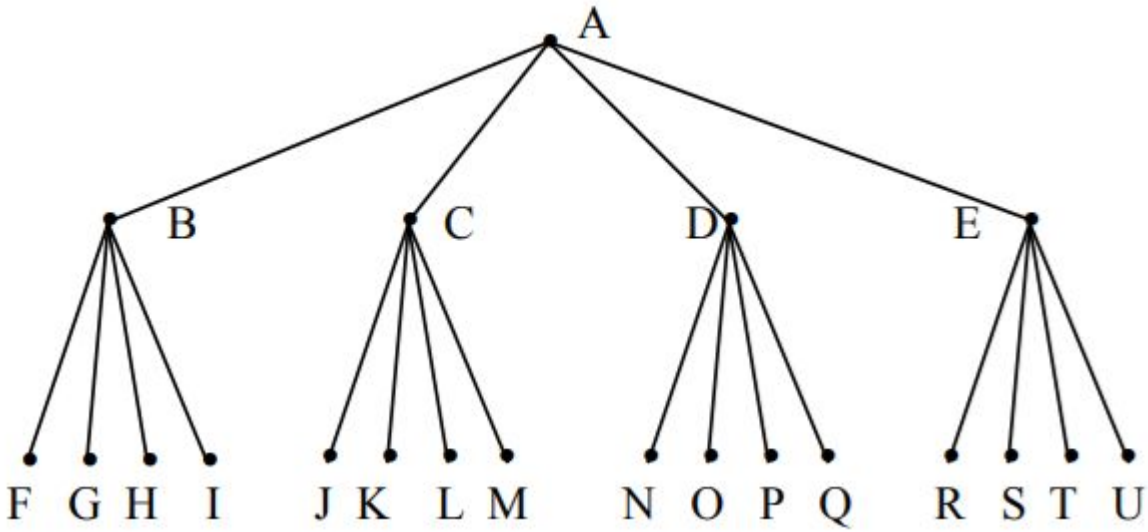
BUSQUEDA DE COSTO UNIFORME (UCS)

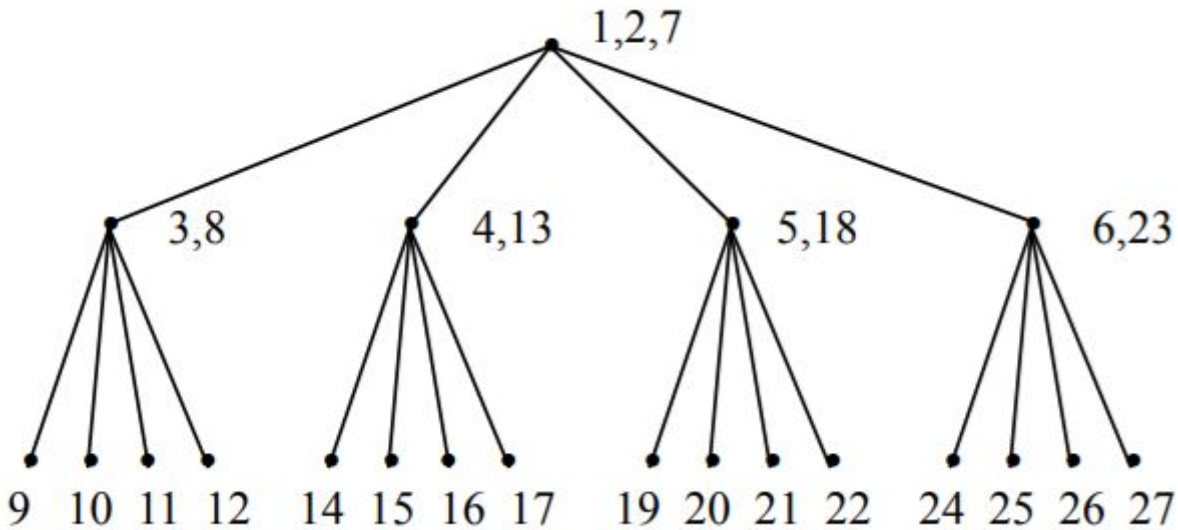


Búsqueda en profundidad iterativa

- Combina la complejidad espacial de la DFS con la optimalidad de BFS.
- Realiza búsquedas en profundidad sucesivas con un nivel de profundidad máximo acotado y creciente en cada iteración.
- Así se consigue el comportamiento de la búsqueda primero en anchura pero sin su coste espacial, ya que la exploración es en profundidad.
- Los nodos se regeneran a cada iteración
- Permite evitar los casos en que DFS no acaba (ramas infinitas).
- En la primera iteración la profundidad máxima será 1 y este valor irá aumentando en sucesivas iteraciones hasta llegar a la solución.
- Para garantizar que el algoritmo acabe si no hay solución, se puede definir una cota máxima de profundidad en la exploración.

Búsqueda en profundidad iterativa





Iteración	nodos
1	1
2	2,3,4,5,6
3	7, … 27

Búsqueda en profundidad iterativa

```
Algoritmo Búsqueda en profundidad iterativa (límite: entero)
    prof=1; Est_abiertos.inicializar()
    Mientras no es_final?(Actual) y prof<limite hacer
        Est_abiertos.insertar(Estado inicial)
        Actual= Est_abiertos.primero()
            Mientras no es_final?(Actual) y no Est_abiertos.vacía?() hacer
                Est_abiertos.borrar_primero()
                Est_cerrados.insertar(Actual)
                si profundidad(Actual) ≤ prof entonces
                    Hijos= generar_sucesores(Actual)
                    Hijos= tratar_repetidos(Hijos, Est_cerrados, Est_abiertos)
                fsi
                Est_abiertos.insertar(Hijos)
                Actual= Est_abiertos.primero()
            fMientras
            prof=prof+1
            Est_abiertos.inicializar()
    fMientras
fAlgoritmo
```

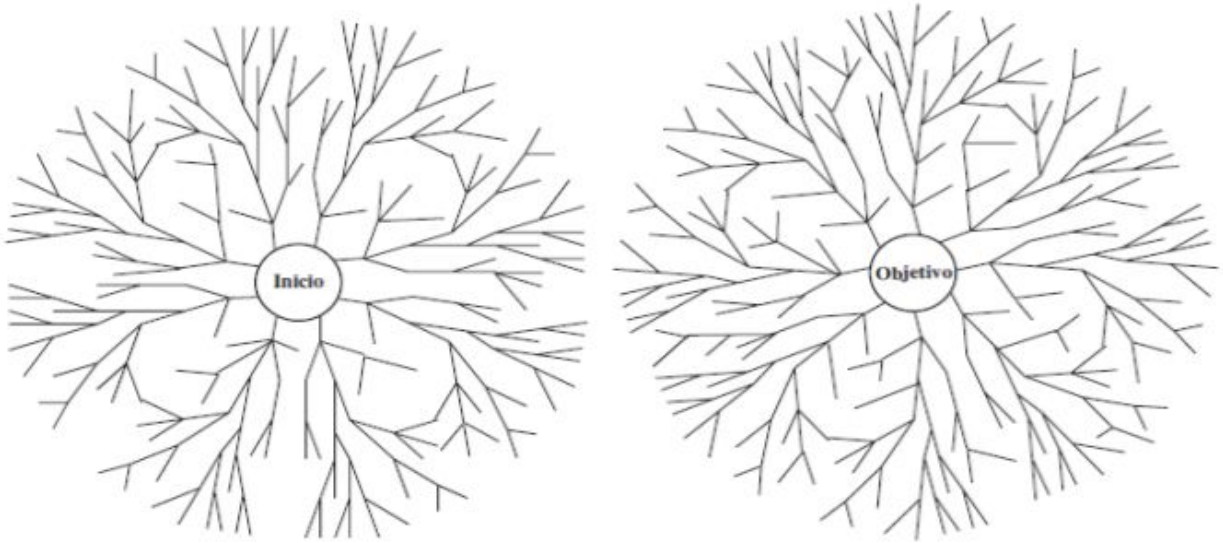
Búsqueda en profundidad iterativa

- Complejidad en tiempo O(r p), donde:
  - r: factor de ramificación.
  - p: profundidad de solución.
- Complejidad en espacio, O(rp).
- Es completa
- Es óptima. Obtiene una solución con el mínimo número de operadores.

Variantes de los algoritmos vistos

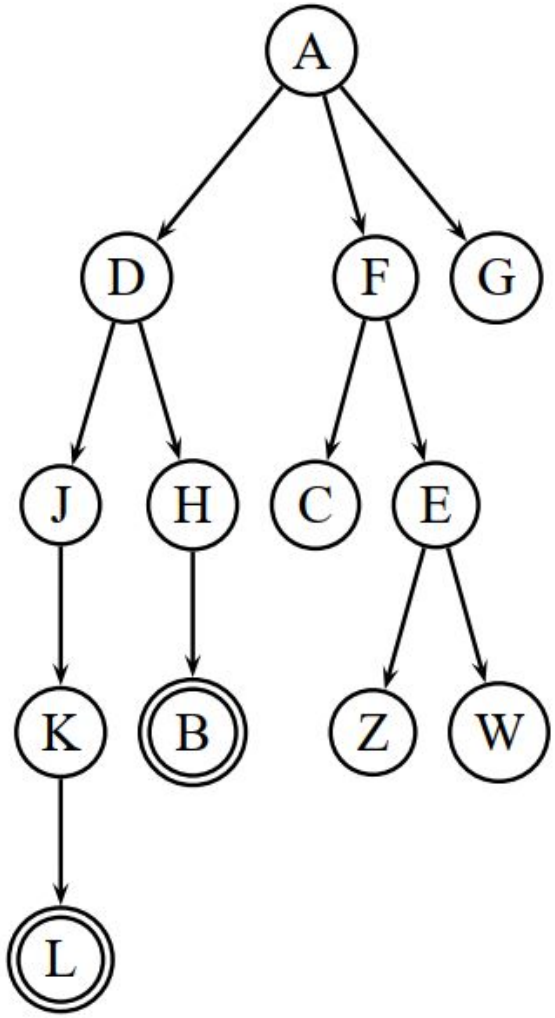
Algunas variantes de los algoritmos vistos para búsqueda No informada:

- Búsqueda bidireccional: conocida la meta, buscamos alternativamente desde salida y desde meta hasta encontrar un enlace.



Ejercicio 1 - Recorridos

Dado el árbol de la figura, donde B y L son los 2 únicos nodos meta y A es el nodo inicial, indica en qué orden se visitarán los nodos, distinguiendo en cada paso nodo actual y nodos en la lista de abiertos, para los algoritmos primero en anchura y primero en profundidad.



- Usa https://visualgo.net/en/dfsbfs para dibujar el grafo y simular las búsquedas

- Ver detalle en aula virtual

Ej. 2: Acertijo de las jarras

Descripción:

Tenemos dos garrafas, una de 4 litros de capacidad y otra de 3, ambas sin marcas de medición. Disponemos de una fuente para llenar las garrafas. Averiguar cómo se puede lograr tener exactamente 2 litros de agua en la jarra de 4 litros de capacidad.

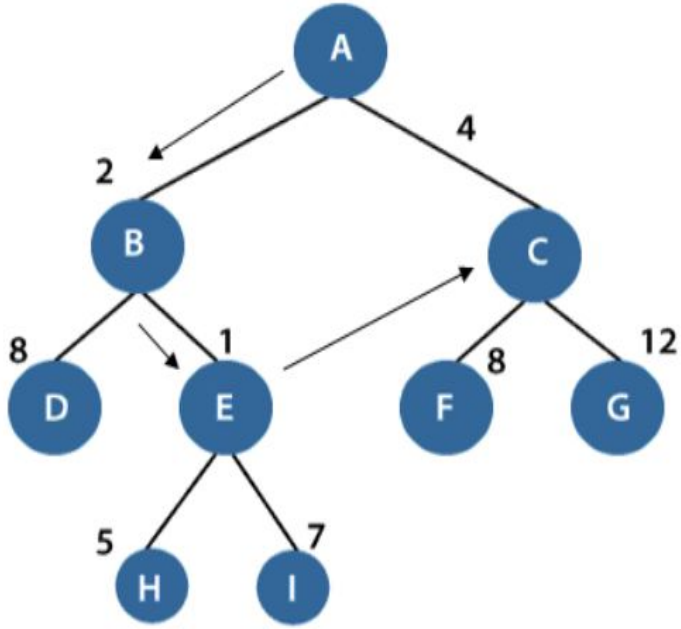
Ver detalle en aula virtual

Búsqueda informada

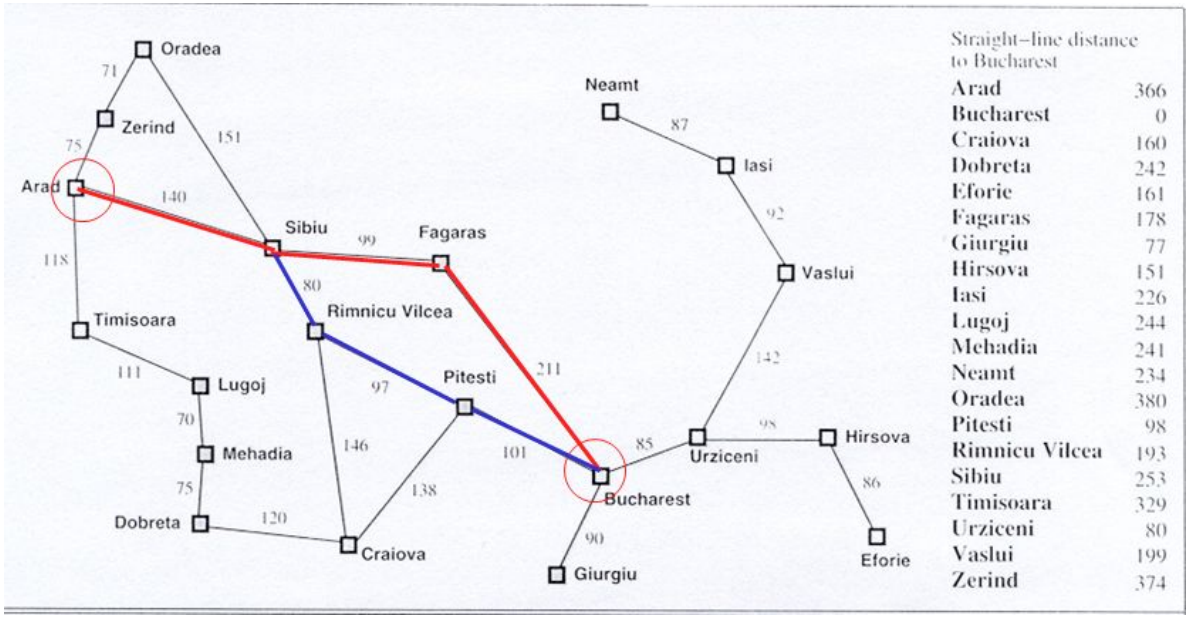
- Usaremos el conocimiento sobre el problema que queremos resolver para tratar de reducir su coste temporal, exponencial en la búsqueda no informada, tratando de minimizar el número de nodos a visitar.
- **Inconveniente**: Este conocimiento no será exportable a otros problemas.
- Necesitaremos una estimación para calcular el coste de cada paso o la distancia al objetivo.
- No se garantiza encontrar una solución, ni que sea la mejor.
- Veremos los siguientes algoritmos de búsqueda informada:
  - Búsqueda primero el mejor (Best First)
  - A*
  - IDA*

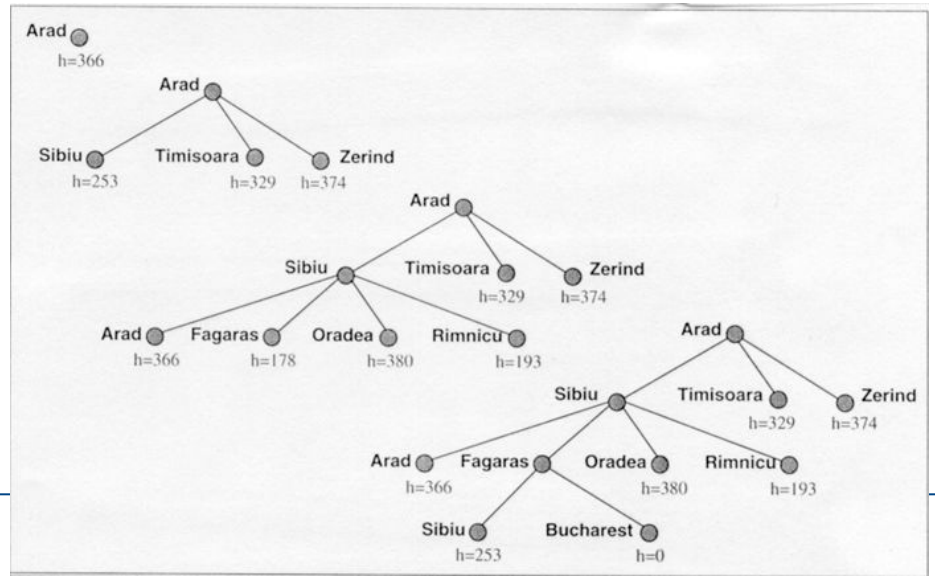
“Greedy” Primero el Mejor (BF)

- Es la aproximación informada más general.
- Selecciona para su expansión al nodo con la función de evaluación más baja (**menor distancia ESTIMADA al objetivo**).
- La estructura de abiertos es una cola con prioridad.
- La función de evaluación no será exacta, y puede dirigir la búsqueda por mal camino.
  - No es óptima.
  - Es incompleta (puede insistir hasta el infinito en caminos que no llevan a la solución).
  - Complejidad temporal y espacial es O (p^m) en el caso peor.



“Greedy” Primero el Mejor (BF)





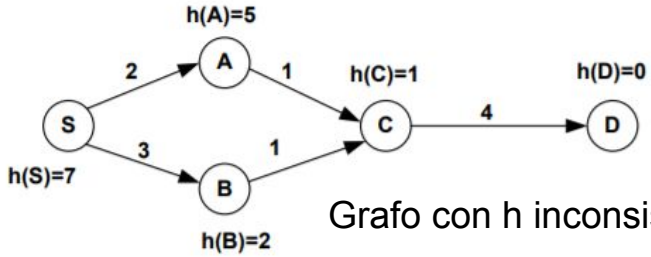
Requisitos Función heurística

Características fundamentales de la función heurística h(n).

- **Debe ser admisible**. Es decir, no puede sobreestimar el coste real. Ejemplos:
  - Distancia en línea recta (euclídea).
  - Distancia Manhattan (suma distancias x,y en un eje de coordenadas).

Toda heurística consistente es admisible, pero una heurística admisible podría no ser consistente.

- **Debe ser consistente (monótona)**, es decir, con
  c(): el coste de una acción
  n: nodo actual
  n': nodo resultante de una acción
  Se cumple siempre que **h(n) ≤ c(n, a, n') + h(n')**
  En cristiano: La heurística de un nodo n tiene que ser menor o igual al coste de ir desde ese nodo a otro nodo n' aplicando una acción a, sumado al coste de la misma heurística en ese nodo n'.
  Si h(n) es consistente, no es necesario el tratamiento de nodos cerrados



Grafo con h inconsistente

Algoritmo A*

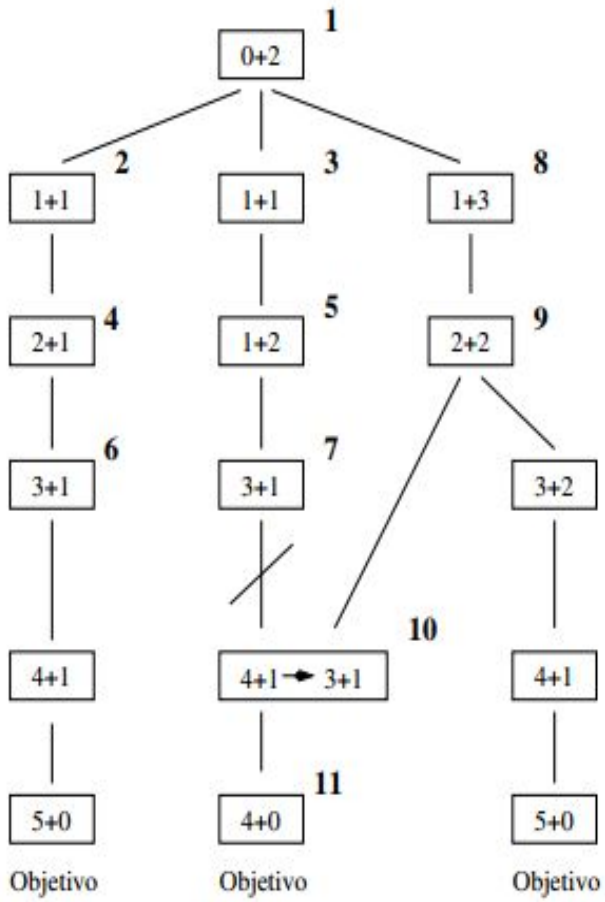
- Utiliza el algoritmo primero el mejor pero aplicando una función de evaluación

  $$f(n) = g(n) + h(n)$$

- g(n) nos da el coste real **ACUMULADO** del camino desde el nodo inicio al nodo n.
- h(n) es el coste **ESTIMADO** del camino más barato desde n hasta el objetivo.

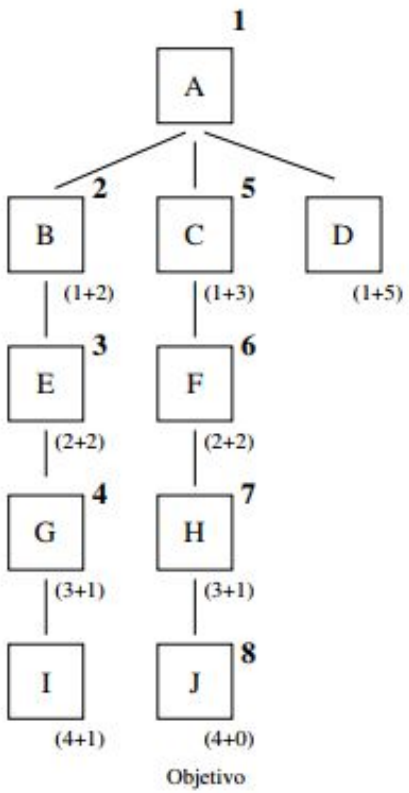
La función h corresponde a la función heurística.

- A igual valor de f los nodos con h más pequeña se explorarán antes, a igual h se considerarán en el orden en el que se introdujeron en la cola.

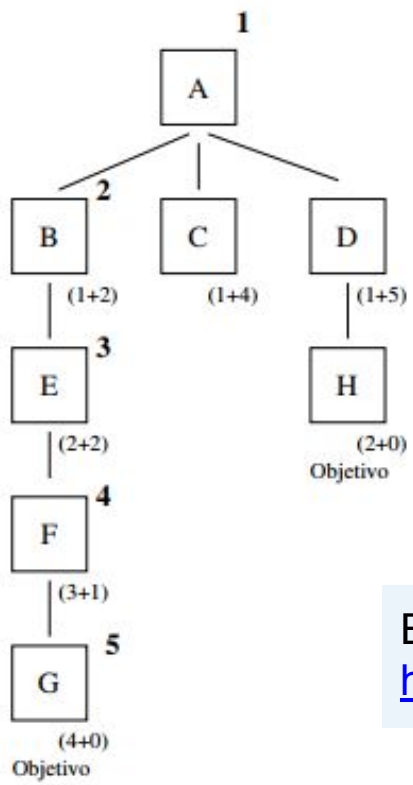


Algoritmo A*

Es óptimo si h(n) es siempre admisible (si no sobreestima el coste del camino de ningún nodo al objetivo).



Recorrido con h(n) admisible



Recorrido con h(n) NO admisible

Problemas o limitaciones:

- Tiende a UCS si h(n) es demasiado optimista.
- Complejidad espacial prohibitiva para espacios de búsqueda grandes o de ramificación alta. En el peor caso, exponencial.

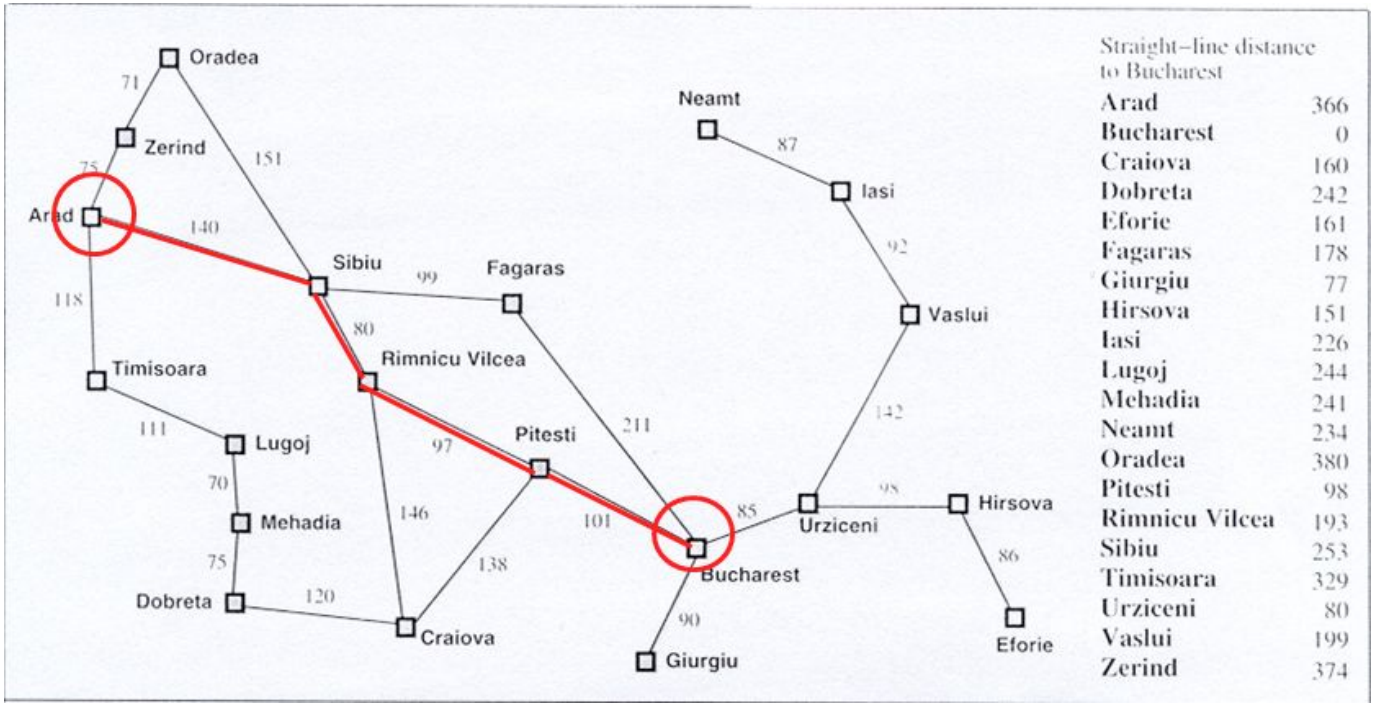
En Vídeo:
https://www.youtube.com/watch?v=1gszEk8rUS4

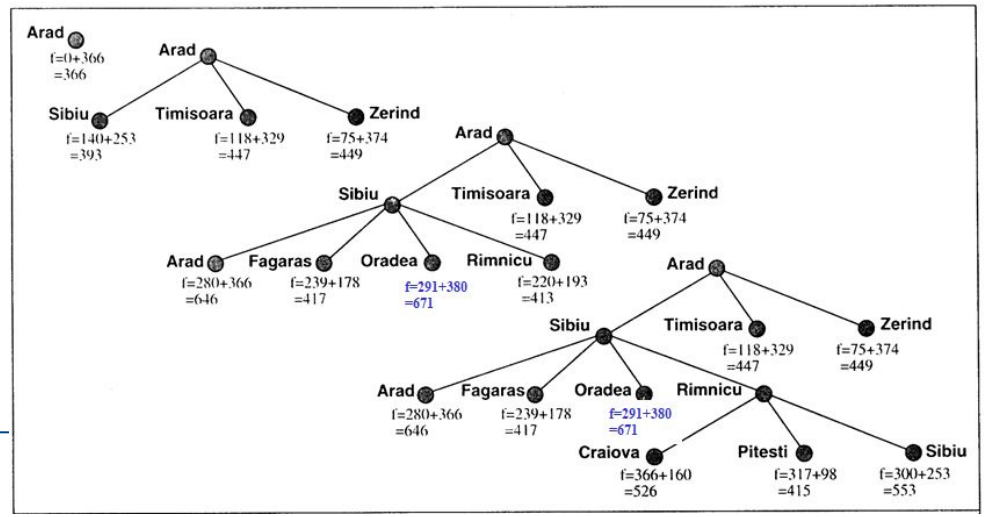
Algoritmo A*

```
Algoritmo: A*
Est_abiertos.insertar(Estado inicial)
Actual ← Est_abiertos.primero()
mientras no es_final?(Actual) y no Est_abiertos.vacía?() hacer
        Est_abiertos.borrar_primero()
        Est_cerrados.insertar(Actual)
        hijos ← generar_sucesores (Actual)
        hijos ← tratar_repetidos (Hijos, Est_cerrados, Est_abiertos)
        Est_abiertos.insertar(Hijos)
        Actual ← Est_abiertos.primero()
fin mientras
FIN
```

- Si es un nodo repetido que está en la estructura de nodos abiertos
  - Si su coste es menor sustituimos el coste por el nuevo, esto podrá variar su posición en la estructura de abiertos
  - Si su coste es igual o mayor nos olvidamos de ese camino.
- Si es un nodo repetido que está en la estructura de nodos cerrados
  - Si su coste es menor reabrimos el nodo insertándolo en la estructura de abiertos con el nuevo coste, no hacemos nada con sus sucesores, ya se reabrirán si hace falta
  - Si su coste es mayor o igual nos olvidamos del nodo

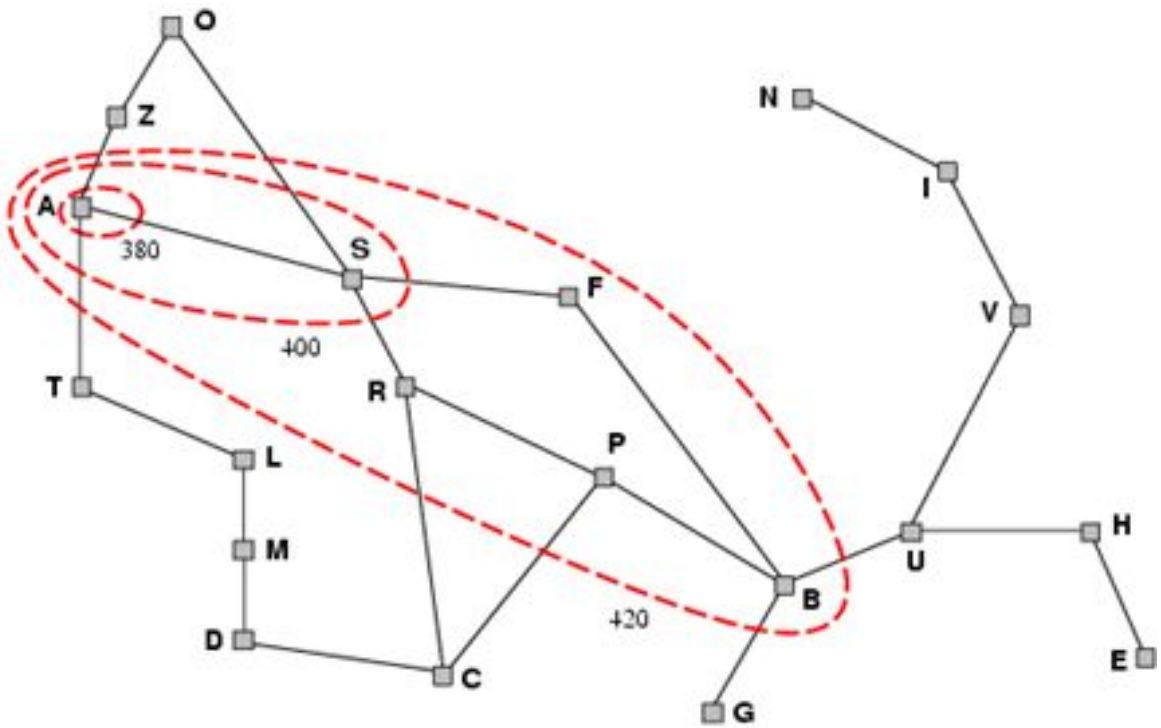
Algoritmo A*


Oradea
Zerind
Arad
Sibiu
Fagaras
Timisoara
Rimnicu Vilcea
Lugoj
Pitesti
Mehadia
Dobreta
Craiova
Neamt
Iasi
Vaslui
Urziceni
Hirsova
Bucharest
Giurgiu
Eforie
71
75
151
140
118
99
80
111
97
70
146
101
211
138
75
120
87
92
142
98
85
86
90
Straight-line distance to Bucharest
Arad 366
Bucharest 0
Craiova 160
Dobreta 242
Eforie 161
Fagaras 178
Giurgiu 77
Hirsova 151
Iasi 226
Lugoj 244
Mehadia 241
Neamt 234
Oradea 380
Pitesti 98
Rimnicu Vilcea 193
Sibiu 253
Timisoara 329
Urziceni 80
Vaslui 199
Zerind 374


Arad
f=0+366
=366
Sibiu
f=140+253
=393
Timisoara
f=118+329
=447
Zerind
f=75+374
=449
Arad
f=280+366
=646
Fagaras
f=239+178
=417
Oradea
f=291+380
=671
Rimnicu
f=220+193
=413
Craiova
f=366+160
=526
Pitesti
f=317+98
=415
Sibiu
f=300+253
=553

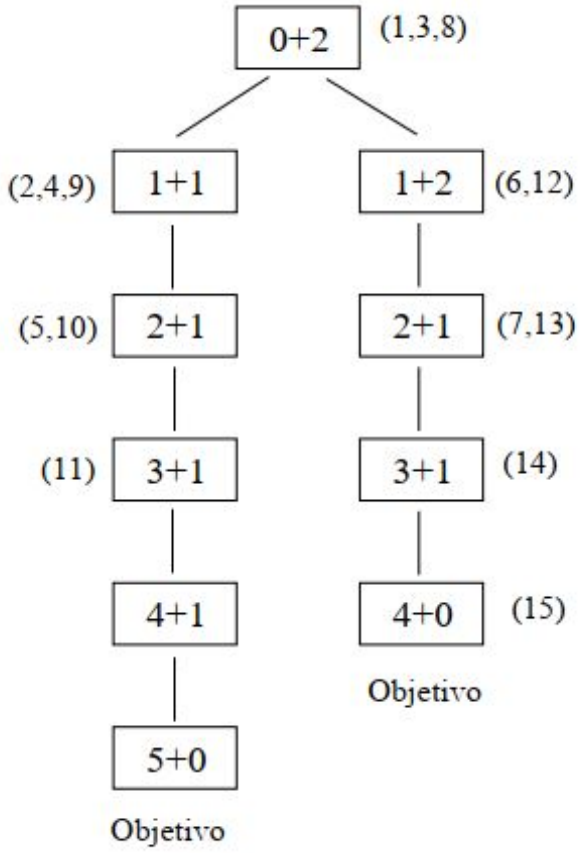
Algoritmo A* - Contorno

- A* expande nodos siguiendo un orden creciente de la función f(n). Esto nos da sucesivos “contornos” de amplitud creciente en cada paso.
- Con h(n) = 0 los contornos serían totalmente circulares.
- Cuanto mejor sea la heurística h(n), más “ovalado” será cada contorno. Se expandirán menos nodos.



Algoritmo IDA* (Iterative Deepening A*)

- Búsqueda iterativa en profundidad, pero usando el valor de la función f como límite a aumentar en cada iteración.
- La complejidad espacial es lineal, como en DFS.
- Por contra, expandimos nodos ya visitados en cada iteración.



Algoritmo IDA* (Iterative Deepening A*)

```
Algorithm: IDA* (limite entero)
prof ← f(Estado inicial)
Actual ← Estado inicial
while no es_final?(Actual) y prof<limite do
    Est_abiertos.incializa()
    Est_abiertos.insertar(Estado inicial)
    Actual ← Est_abiertos.primero()
    while no es_final?(Actual) y no Est_abiertos.vacía?() do
        Est_abiertos.borrar_primero()
        //Est_cerrados.insertar(Actual)
        Hijos ← generar_sucesores (Actual, limite)
        Hijos ← tratar_repetidos (Hijos, Est_cerrados,Est_abiertos)
        Est_abiertos.insertar(Hijos)
        Actual ← Est_abiertos.primero()
    Fin while
    prof ← prof+1
Fin while
Fin
```

Sólo genera aquellos sucesores con una f menor a la del límite impuesto

Si se tratan repetidos en cerrados, el ahorro en espacio es nulo. Si h es consistente, no hace falta hacerlo en cerrados

La estructura de los nodos abiertos es una pila, como en DFS

Búsqueda heurística . Otros algoritmos

Variantes de los algoritmos presentados:

- Primero el mejor recursivo: versión recursiva del algoritmo primero el mejor, lo que lo dota de coste espacial lineal.

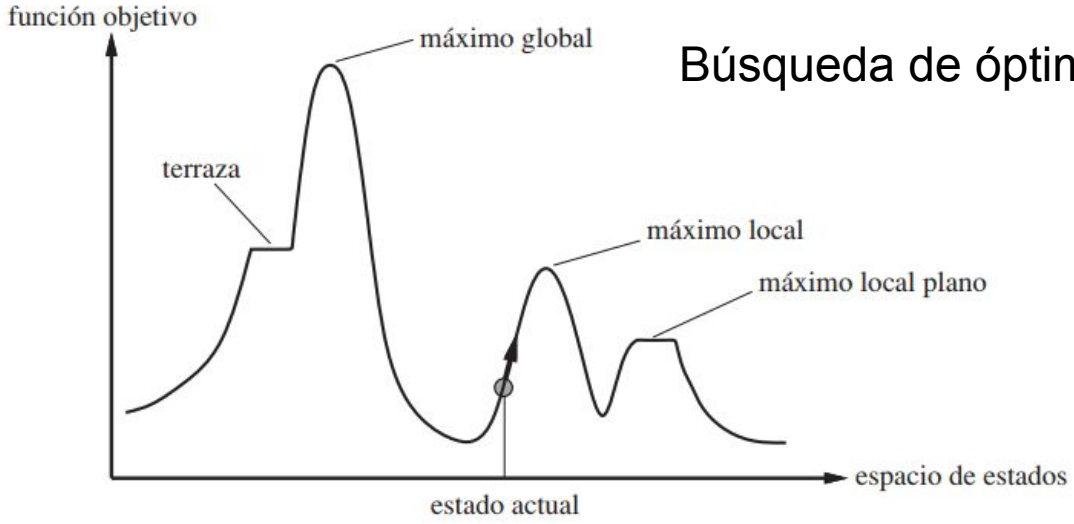
- A* con memoria acotada (Simplified Memory Bounded A): Variante de A* que utiliza un límite de nodos a almacenar en memoria, podando las ramas menos prometedoras.

Búsqueda heurística. Ejercicios

- Ejercicios 2.3 y 2.4 Aula Virtual

Búsqueda Local

- En ocasiones, el camino hacia la solución es irrelevante, o hallar la solución óptima es inviable.
- Partimos de una configuración inicial (a menudo aleatoria) y le aplicamos operadores para tratar de acercarnos lo máximo a una solución óptima en el menor tiempo posible.
- Se mueve entre estados vecinos (cambios pequeños entre estados).
- Suelen acabar en soluciones locales, no óptimas.
- Usaremos la función heurística para **estimar** la calidad de cada solución, o para podar ramas poco prometedoras.
- La heurística ya no tiene restricciones (no tiene que ser admisible o consistente), sólo debe comparar soluciones.



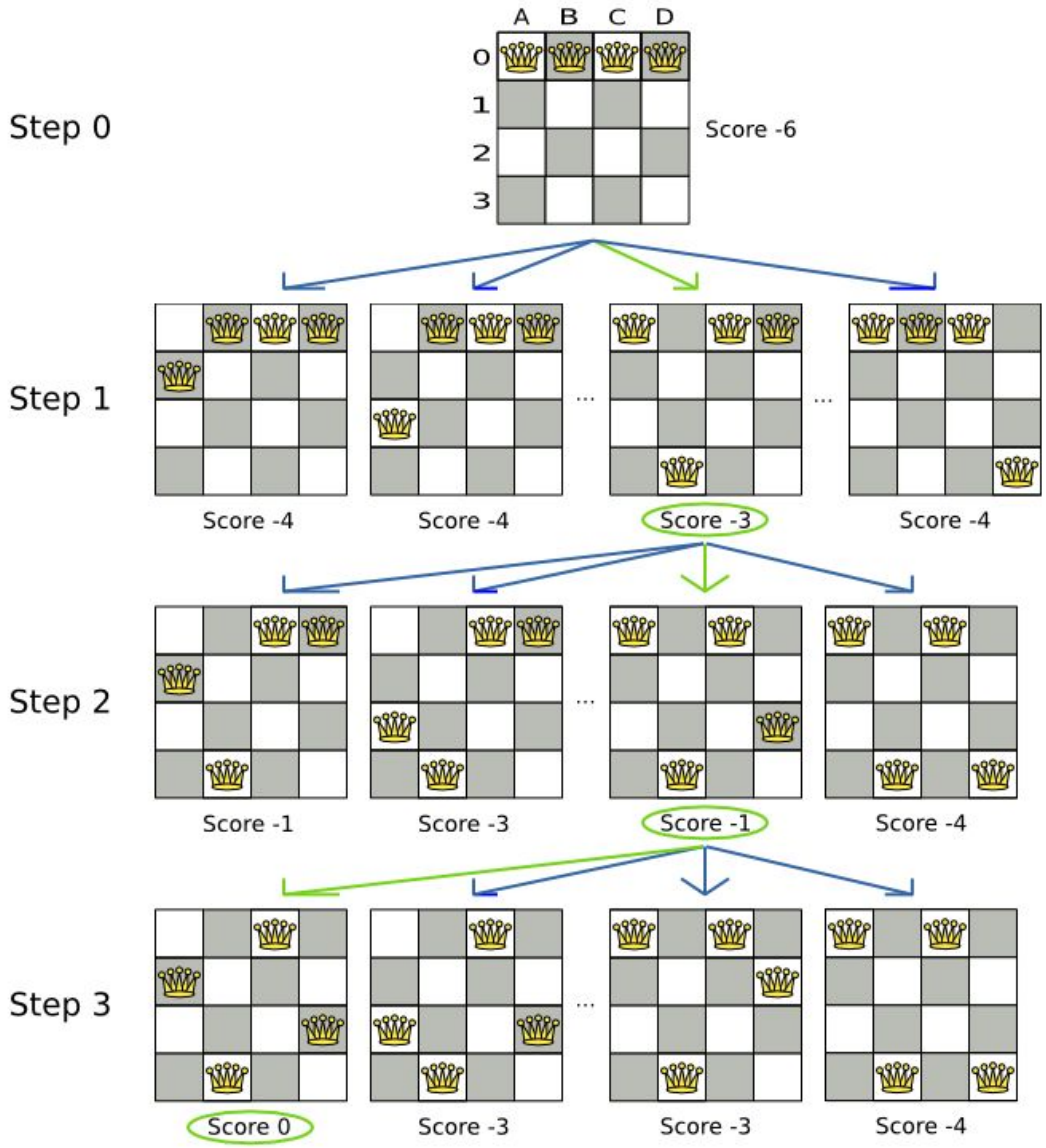
Búsqueda de óptimos en funciones

Búsqueda Local

Aplicaciones empresariales (Generalmente problemas de optimización):

- diseño de circuitos integrados
- disposición del suelo de una fábrica
- programación del trabajo en tiendas
- programación automática
- optimización de redes de telecomunicaciones
- gestión de carteras

Búsqueda Local

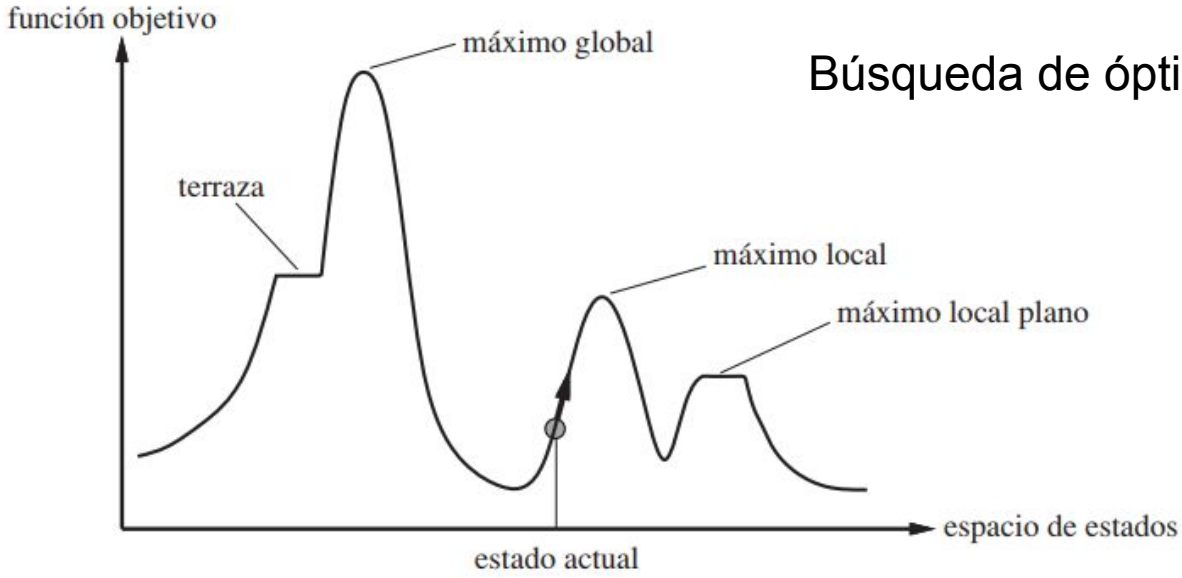


Búsqueda Local

Ventajas respecto a informada/no informada:

- El coste espacial se mantiene constante (uno sólo o pocos estados en memoria).
- Baja complejidad temporal.
- Aplicables a espacios de búsqueda infinitos y continuos.

Desventaja fundamental: No garantiza mejorar la solución (terrazas), ni encontrar la óptima (máximos locales).



Búsqueda de óptimos en funciones

Ascensión de Colinas (Hill-climbing)

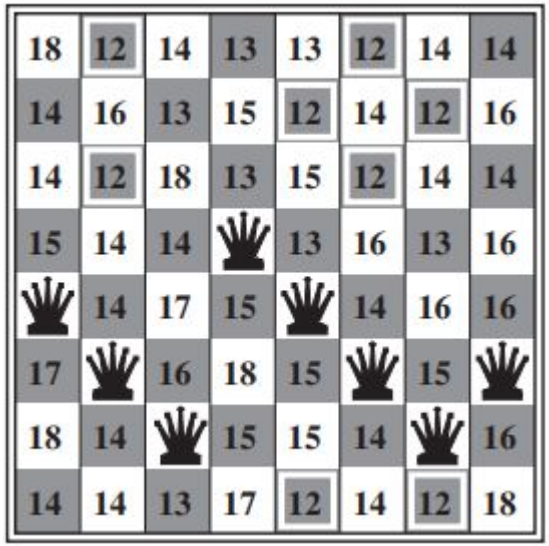
- **Escalada simple:** Opta por el primer estado vecino que suponga una mejora respecto al estado actual.
- **Escalada por máxima pendiente** (steepest-ascent hill climbing): Se selecciona el mejor movimiento (no el primero de ellos) que suponga mejora respecto al estado actual.

```
Algoritmo Steepest Ascent Hill Climbing
    Actual ← Estado_inicial
    fin ← falso
    mientras no fin repetir
        Hijos ← generar_sucesores(Actual)
        Hijos ← ordenar_y_eliminar_peores(Hijos, Actual)
        si no vacio?(Hijos) entonces
            Actual ← Escoger_mejor(Hijos)
        sino
            fin ← cierto
        fin si
    fin mientras
FIN
```

- **Completo:** No (puede quedarse en óptimos locales que no sean soluciones).
- **Óptimo:** No, no garantiza encontrar el óptimo global.
- **Complejidad temporal** lineal o logarítmica.
- **Complejidad espacial** constante.

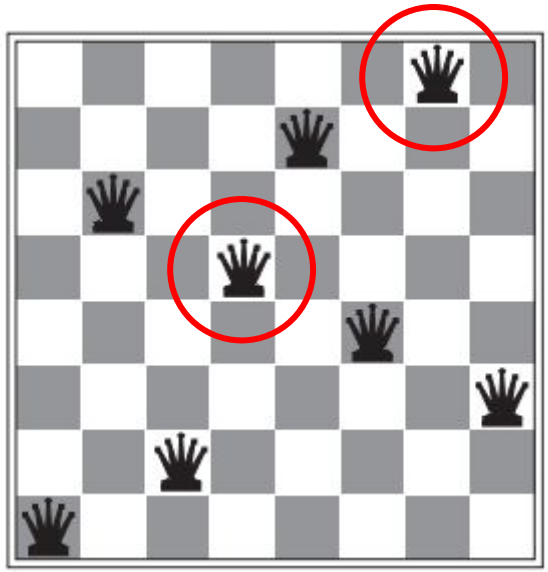
Ascensión de Colinas (Hill-climbing)

Con Hill Climbing, en el problema de las 8 reinas, se alcanza sólo un 14% de veces un estado solución (no local).



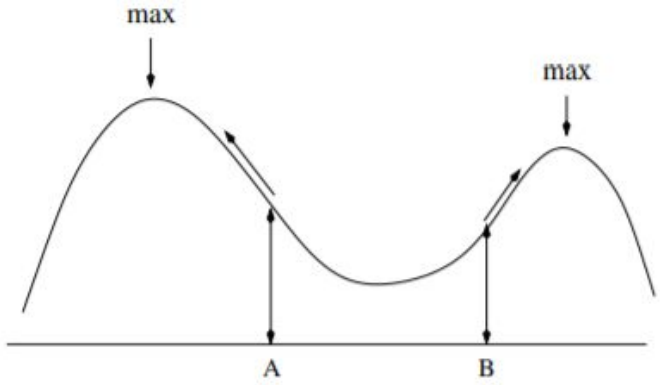
Ejemplo de Configuración Inicial para el problema de las 8 reinas. Con h = 17 (pares de reinas que se matan).

Un mínimo local en el espacio de estados de las 8-reinas, con h = 1, pero en que cada sucesor tiene peor evaluación.



Ascensión de Colinas: variantes

Ascensión con reinicio aleatorio (random restarting hill climbing): probamos distintos puntos de inicio.



Búsqueda por haz local (local beam search). Guardamos los N mejores nodos en cada iteración o nivel (el mejor, el segundo mejor… el n-esimo mejor), siguiendo hasta que ningún sucesor suponga una mejora.

Ascensión de colinas estocástica. Escoge aleatoriamente de entre los movimientos ascendentes mejores; la probabilidad de selección puede variar con la pendiente del movimiento ascendente.

Ascensión de colinas de primera opción: genera sucesores aleatoriamente hasta que se genera uno que es mejor que el estado actual. Buena estrategia cuando un estado tiene muchos (por ejemplo, miles) sucesores.

Temple simulado (simulated annealing)

- Un algoritmo que sólo asciende, probablemente será incompleto, y otro aleatorio resulta a menudo ineficaz.
- Combinaremos ambas estrategias. Escogemos un movimiento aleatorio:
  - Si supone una mejora, nos quedamos con él para seguir iterando.
  - Si empeora la situación, nos quedamos con él con probabilidad exponencial:

$$e^{-E/T}$$

E (Energía): La “maldad” del movimiento (cuánto empeora la situación).
T: Temperatura (descendiente en cada iteración).

- Los malos movimientos se escogerán con mayor probabilidad en las primeras iteraciones.

Ejemplo en acción:
https://toddwschneider.com/posts/traveling-salesman-with-simulated-annealing-r-and-shiny/

Temple simulado (simulated annealing)

Algoritmo: Simulated Annealing

Partimos de una temperatura inicial
mientras la temperatura no sea cero hacer
    // Paseo aleatorio por el espacio de soluciones
    para un numero prefijado de iteraciones hacer
        Enuevo ← Genera_sucesor_al_azar(Eactual)
        ΔE ← f(Eactual) - f(Enuevo)
        si $\Delta E > 0$ entonces
            Eactual ← Enuevo
        sino
            con probabilidad $e^{\Delta E/T}$: Eactual ← Enuevo
        fin si
    fin para
    Disminuimos la temperatura
fin